Session 4: Advanced Claude Code

Skills, MCPs, and Subagents

Brady Allardice

UPF & IBEI

Where We Are

Session 1: What Claude Code can do. How it works. CLAUDE.md and plan mode.

Session 2: The core workflow. Data cleaning, estimation, robustness, LaTeX tables.

Session 3: Git and LaTeX in VS Code. Your paper joins the project.

Session 4: The layer above the core workflow — infrastructure.

Today you will:

1. Migrate to a cleaner repo workflow (a structural fix, not an agent fix)
2. Learn to route a wish — is it a **skill**, **project context**, or **MCP**?
3. Connect two MCPs (Zotero, GitHub) in practice
4. See when to reach for a **subagent** — and when not to

Today is about knowing which tool does real work and which is just novelty.

Check-In

Since Session 3:

- Did the git cycle (commit $\rightarrow$ instruct $\rightarrow$ diff $\rightarrow$ decide) stick?
- Did anyone bring their paper into VS Code? How did it feel?
- Any tool fight back at you — compilation errors, merge conflicts, something else?

Hold onto the merge-conflict stories. We start there.

Agents solve symptoms well.
They do not notice when a problem shouldn't exist.

Fifteen minutes. One migration. The meta-lesson for the rest of the session.

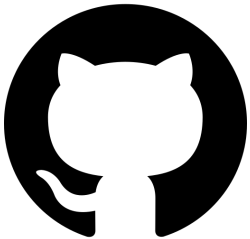
First: What Is GitHub?

GitHub is where code lives on the internet.

Every project is a *repository* — a folder with all its files plus the full history of every change.

Two copies of everything:

- **Remote** — the shared version on `github.com`
- **Local** — the version on your machine



The short version

GitHub hosts the *shared* copy. You work on your *local* copy. Git keeps them in sync.

The Vocabulary

Five words that will keep coming back today:

- `clone` — make your first local copy of a remote repo
- `pull` — fetch the latest changes *from* the remote into your local copy
- `push` — send your local changes *to* the remote
- **branch** — a parallel line of work inside the repo; changes on a branch don't affect anyone else until it's merged
- **pull request (PR)** — a proposal to merge one branch into another, with a page on GitHub for review and discussion

The flow

`clone` once. `pull` before you start. Make a **branch**, commit, `push`, open a **PR**. Someone reviews and merges.

The Course Repo

Our materials live in one GitHub repo. Everything I push goes there; everything you pull comes from there.

Let's look at the page together. Things to notice:

- The **file tree** — `session_1/`, `session_2/`, ..., `session_4/`
- The **README** — what the repo is and how to use it
- The **commit history** — every change I've made, with a message
- The **branches** — `main` is the shared one; others are work-in-progress

Why this matters for what's next

The issue we hit last session lives exactly here: you and I were both editing files in the *same* repo. To explain what went wrong, we need the picture of where things live.

The Problem We Had

Every session so far, someone has hit a merge conflict.

What we did about it:

- Deleted the local repo and recloned
- Asked Claude Code to resolve the conflict markers
- Copied work to the Desktop, wiped the directory, copied back

That worked. Everyone got unstuck. But it was a symptom fix.

The Root Cause

You were editing files *inside* the shared course repository. Every time I pushed new material, git had to reconcile your changes with mine. Conflicts were guaranteed by the structure, not by accident.

The Structural Fix

New layout on my side: each session is a self-contained folder.

```
AIAgentsCourse/
  session_1/
  session_2/
  session_3/
  session_4/ <- today's materials live here
  ...
```

New workflow on your side: copy the session folder out of the repo into your own workspace. Work there. The shared repo stays read-only for you.

What this changes

I can push new material anytime without breaking anything in your workspace. You can edit, break, delete, and redo without worrying about the shared state. No more conflicts.

The Three-Step Ritual

Do this at the start of every session from now on:

```
# 1. Pull the latest course repo
cd ~/AIAgentsCourse && git pull

# 2. Copy today's session folder into your workspace
cp -r session_4/ ~/my_workspace/session_4/

# 3. Open your copy in VS Code and work there
cd ~/my_workspace/session_4 && code .
```

We do it together now, live. Five minutes. Everyone ends up in the same clean state before we move on.

If you had in-progress work inside the course repo, copy it into `~/my_workspace/` too. I will help you sort out what to keep.

The Meta-Lesson

Agents solve symptoms well. They do not notice when a problem shouldn't exist.

Claude Code could have resolved merge conflicts forever. It was good at it. What it would not have done is stop and say: *"Why are you editing the shared repo in the first place?"*

Your job, not the agent's: when a symptom keeps recurring, ask whether the structure is wrong.

A question to carry with you

When you catch yourself reaching for Claude Code to patch the same thing a third time, pause. Is there a structural fix you're missing?

This lesson lands again when we talk about skills (are you using it, or did you just build it?) and MCPs (does this earn a permanent connection, or is it just novelty?).

A **skill** is a reusable instruction bundle
Claude loads on demand when a task matches.

It codifies a *convention* —

how *you* do a thing correctly —
so you don't re-explain it every session.

Fifty minutes. The centerpiece of today.

Starting With an Admission

I have built four skills:

- `auto-commit`
- `research-docs`
- `robustness-checks`
- `spec-validator`

I barely use any of them.

I'm not teaching you an aspiration. I'm teaching you what I learned from building four things I don't use.

Why They Didn't Stick

Looking back, what these four have in common:

- Built *on spec* — "this would be good to have" — not in response to pain
- Flavored toward *generic discipline* — "good research practice"
- Not tied to a specific failure mode I actually hit
- Each could be replaced by a prompt the moment I needed it

The pattern is predictable once you see it. A skill built on an idea of how research *should* go — rather than on a pain I kept hitting — doesn't survive contact with actual work.

The lesson

Most skills people build get abandoned. The ones that survive are tied to concrete, repeated failure modes or stable conventions.

Before You Build a Skill, Ask Where It Belongs

Three things look alike but are different tools:

1. **Skills** — reusable instruction bundles, loaded on demand
2. **Project context** — a CLAUDE.md inside your paper's directory
3. **MCPs** — connections to external systems

Most wishes of the form "I wish Claude knew X" route to one of these three.

Knowing which one saves you from building a skill that should have been something else. It also tells you when a skill is *too narrow* a tool for the job.

The Routing Rule

Skills $\rightarrow$ what's stable *about you*

Projects $\rightarrow$ what's stable *within a paper*

MCPs $\rightarrow$ what's *live*

If X changes when the project changes, it's not a skill.
If X is alive in an external system, it's not a skill either.

The Rule, Applied

What you want Claude to know	Where it belongs
Your LaTeX table conventions	Skill (stable about you)
The variables & specs of your current paper	Project context (within a paper)
Your Zotero library	MCP (live external state)
Your writing voice	Skill (stable about you)
What your current paper argues	Project context (within a paper)

Why this matters

Skills are a narrower tool than most people assume. That's a feature — narrower tools are easier to use well.

Within Skills: Two Different Jobs

Verification skills

Fire *during or after* work to check it.

Examples: LLM-mistake checker, sanity-check validator, replication auditor.

Earn keep by: catching real problems, even if used infrequently.

Production skills

Fire *during the making* of something to shape it.

Examples: LaTeX table skill, writing-style skill.

Earn keep by: frequency — every table, every document.

Why the distinction matters

"Is this worth it?" has a different answer for each. Verification skills need to catch high-cost errors. Production skills need to fire often.

The Heuristic

Skills are for **conventions**

you hold about how to do
a thing correctly.

Conventions about table formatting. About sanity-checking data.
About responding to reviewers. About engaging with the literature.

If what you want to codify isn't a convention, it's probably not a skill.

The One I Actually Use: LaTeX Tables

Failure mode it addresses:

Every table I generate needs two or three rounds of formatting before it's submission-ready. Same rounds, every time.

Conventions the skill encodes:

- Significance notation (stars vs. p-values; thresholds; footnote placement)
- Standard-error style (robust, clustered, which level to display)
- Table-notes format (variable definitions, sample restrictions)
- Override conditions for journal-specific requirements

Why this is a convention, not a prompt

Re-specifying these rules every time is expensive. The conventions are stable. They're about me, not about the paper.

Where It Helped (This Past Week)

Concrete example:

Regenerated three robustness tables after a referee asked for a different clustering structure. With the skill, each table came out consistent with my house style on the first pass — notes, significance stars, SE labels all in place.

What the skill saved:

- Roughly 15 minutes per table of formatting fixes
- Zero context-switching between "what did I format like last time?" and the actual analysis
- Consistency across tables in the same paper — something I'd drift on manually

Small per-use savings. But tables compound — every paper has many.

Where It Fell Short (And What I'd Change)

Where it fired wrong:

- Over-eager on notes — added full table notes when I wanted the slide version
- Didn't know the target journal's specific significance conventions until I told it

What I'd change:

- Explicit trigger conditions: paper conventions vs. slide conventions
- A one-line journal-override mechanism at the top of the skill

The point

Skills are empirical artifacts. You build them, fail at them, refine them. The LaTeX skill is on version three.

Four More I've Built. Not Yet Trialed.

Explicitly a different bucket from the LaTeX skill:

1. **LLM-mistake checker** — catches the class of errors Claude systematically makes (silent NaN drops, wrong merge keys, ignored clustering)
2. **Sanity-check validator** — fires on freshly-loaded data to verify ranges, types, and counts match expectations
3. **Testing-philosophy skill** — prevents over-mocked tests that pass but don't catch real bugs
4. **Paragraph-structure audit** — one claim per paragraph, mis-cited claims, smuggled assumptions, weak transitions. The integrated version queries Zotero for candidate citations when claims are uncited.

These I built. I'll trial each once before class. But none are yet validated by real use.

Why I Think These Might Stick

What's different from the earlier four:

- Each is tied to a *specific LLM failure mode*, not generic "good practice"
- Each is a *verification skill* — fires when there's a concrete question ("is this code right?" "is this data clean?"), not when I'm ambiently "doing research"
- The trigger condition for each is sharp — I can picture the moment I'd reach for it

What I'm waiting to learn:

Whether they fire at the right moments — or whether the description matching misses the cases that actually matter.

Place your bets

Which of the four survives three months? I'll report back in a later class.
Skill adoption is empirical, not theoretical.

Where Skills and MCPs Compose

The paragraph-structure skill is the first one you've seen that invokes an MCP.

Alone:

- *Skill only:* "This claim needs a citation. Figure one out."
- *MCP only:* "Here's your Zotero library." But nothing knows when to search.

Composed:

- The skill knows *when* a claim is mis-cited or uncited
- The MCP returns actual papers from your library that would support it
- The MCP constrains the skill to real papers — no hallucinated citations

The pattern

A skill knows *when* to act; an MCP provides *live data*. Together, they do what neither can alone.

Exercise: Generate a Candidate (10 min)

Pairs. Each of you names one concrete moment, in the last two weeks, where:

- You wished Claude had already known something, **or**
- You retyped context you'd typed before

Partner helps you decide:

1. Is this a **skill**, **project context**, or **MCP**?
2. If a skill: is it a **convention**?
3. Is it **verification** or **production**?
4. What **specific failure mode or convention** does it address?

Deliverable

One candidate skill, with a one-sentence justification.

- **What:** a name and a one-sentence description
- **Why it's a skill:** not a project artifact, not an MCP
- **What it addresses:** a named failure mode or convention

Two or three pairs share out.

The goal

Not that you leave having built a skill. That you leave able to recognize when a candidate is worth building — and when it isn't. So you don't repeat my four-built-none-used pattern.

Skills: The Arc

1. Skills get abandoned when they're built on spec, not on pain
2. Before building, **route:** stable-about-you, stable-within-a-paper, or live?
3. Within skills, ask: **verification** or **production**?
4. Skills are for **conventions** about how to do a thing correctly — not for good intentions
5. Skills are **empirical**: build, fail, refine

The durable takeaway

If what you want to codify is a convention about how *you* do a thing correctly, and it doesn't live inside a paper or an external system — then maybe it's a skill. Otherwise it's something else. And that's a feature.

Connecting Claude Code
to systems outside your project.

From the routing rule: *MCPs are for what's live.*
Here are two live things worth connecting.

Forty-five minutes. Two MCPs. One pair exercise.

What an MCP Is

MCP = Model Context Protocol. An open standard for plugging external tools into Claude Code.

- Each MCP is a separate server.
- Claude Code speaks to it over a standard protocol.
- The server exposes *tools* (like `zotero_search`) that Claude Code can call.

Without MCPs: Claude Code reads and writes files in your project. That's it.

With MCPs: it can query your Zotero library, open a GitHub PR, read a database, post to Slack — anything someone has built a server for.

The framing

MCPs turn Claude Code from a tool that operates on your *files* into a tool that operates on your *workflow*.

The Two-Reason Rule

Once you've routed something to an MCP, ask: which of two reasons?

1. **Extend capability.** Let Claude Code do something it couldn't do at all.
   Example: open a GitHub pull request. Query your Zotero library. Run a SQL query against your database.
2. **Reduce friction at a boundary.** Let it do something you *could* already do, but where context-switching between tools kills the flow.
   Example: inserting citations while drafting. You could look them up in Zotero and paste. But stopping to do that breaks writing flow.

Today I'm showing two specific MCPs I think are worth adopting. There are dozens more. The *framework* is what transfers; the two cases are how you learn it.

Callback to the meta-lesson: MCPs are easy to accumulate. Ask what each one earns.

The Friction Case: Zotero

Drafting with citations normally looks like:

1. Write a sentence that needs a citation
2. Stop. Open Zotero. Search. Find the paper. Copy the cite key.
3. Come back to the editor. Paste. Context-switch back into drafting.
4. Repeat ten times per paragraph.

With the Zotero MCP: "Find me the Smith 2019 paper on voter ID, add it to refs.bib, and cite it here." Claude Code does the search, the BibTeX export, the `\cite{}` insertion — without you leaving the `.tex` file.

Why it earns its keep

Reduce friction at a boundary. You could already do all of this manually. The MCP just removes the tool-switch.

Zotero MCP: Live Demo

A single prompt:

```
Find three recent papers on voter ID laws from my
Zotero library, add them to refs.bib, and draft one
sentence citing each in methods.tex.
```

What you'll see in the tool-call log:

1. `zotero_semantic_search("voter ID laws")` — finds candidates
2. `zotero_get_item_metadata(...)` — pulls BibTeX fields for each
3. `Write refs.bib` — creates or appends to the bibliography
4. `Edit methods.tex` — inserts three sentences with `\cite{...}`

Watch the tool calls, not just the output. Seeing which tools fire tells you what the MCP actually does.

Where Would This Reduce Friction for You?

One minute, with the person next to you:

- Where in your own drafting workflow do you break flow to check Zotero?
- Is it frequent enough that removing the context-switch would actually matter?
- Or is it rare enough that the setup cost isn't worth it?

Installation is homework, not class time

Installing the Zotero MCP is fifteen minutes of fiddling with API keys and configuration files. I am not making you do it in class. Setup guide is in the Session 5 prework.

The Capability Case: GitHub

GitHub MCP lets Claude Code do things you couldn't do by just editing files:

- Open a pull request from a branch you've been working on
- Leave review comments on a coauthor's PR
- Create issues to track tasks you don't want to lose
- Push files to a remote repository

Why now? This morning's Git restructuring put you in a clean state. You are working in your own workspace. You understand the three states: local copy, pushed branch, shared main. The GitHub MCP is what closes the collaboration loop.

Why it earns its keep

Extend capability. "Open a PR" isn't something Claude Code can do without this. It's a genuine new capability, not a friction fix.

The Mental Model

Branches + PRs are the collaboration mechanism. The GitHub MCP is a *helper* on top — not a substitute.

The discipline

Use `git` directly. Use the MCP when you're stuck or when the MCP genuinely saves steps. When things break, you need to understand what's actually happening — and the MCP abstracts that away.

Exercise Setup: Collaboration Pairs (5 min)

Find a partner. You will work as a pair for the next 25 minutes.

Each pair gets: a link to a small template repo with 2–3 editable files and a README explaining the exercise.

The arc:

1. Fork the template (each of you — separate forks)
2. Each makes a change on your own branch
3. Open a PR on your partner's fork
4. Partner reviews and merges

Use Claude Code + the GitHub MCP to do the PR opening and reviewing. Use `git` directly for the commits and branches.

Exercise: The PR Loop (20 min)

Pair workflow:

1. **Fork** the template repo I shared in chat
2. **Clone** your fork to `~/my_workspace/`
3. **Branch:** `git checkout -b my-change`
4. **Edit** one of the files (different files for you and your partner)
5. **Commit + push** the branch
6. **Ask Claude Code** to open a PR using the GitHub MCP:
   "Open a PR from my-change to main on my fork with title X and body Y."
7. **Partner reviews** the PR (also via Claude Code + GitHub MCP) and merges it

Extension, if you finish early: each edit the *same* file, hit a conflict, resolve it together.

I will circulate. Flag me when you're stuck — don't silently spin.

What to Expect

Some pairs will finish. Some will not. Both are fine.

The goal is that *everyone sees the workflow end-to-end*: branch, push, PR, review, merge. Even if you needed help at every step, you will recognize the shape next time.

What will go wrong (for some of you):

- The GitHub MCP will ask for a token you don't have. (I have a fallback.)
- The PR will open against the wrong base branch. (Fixable in the UI.)
- Your partner will merge something you weren't ready for. (That's the workflow.)

You will practice more of this in Session 5. Today: just get the shape in your head.

MCPs: The Meta-Lesson, Again

GitHub MCP is optional infrastructure. The underlying git workflow is not.

If you find yourself reaching for the MCP to fix a conflict — sometimes that's the right move. But sometimes (like this morning's session-folder fix) it's papering over a structural problem.

Before adopting an MCP, ask

1. Does this genuinely extend what I can do, or reduce real friction?
2. Will I invoke it more than three times a month?
3. Am I using it because it helps, or because it's new?

Most MCPs fail question 2. The two today pass all three — *for me*. Your answers may differ.

Know they exist.

Know when to reach for them.
Don't over-invest.

Ten minutes. No hands-on.

What a Subagent Is

Claude Code can spawn another Claude Code instance inside your current session — a subagent.

- It has its own context window
- It gets a scoped task and a scoped tool set
- It returns a summary (not its raw output) to your main conversation

Two values:

1. **Parallelism** — run three searches at once instead of serially
2. **Context hygiene** — a big messy search doesn't pollute your main conversation

Think of it as

Delegating a bounded sub-task to a fresh colleague who doesn't see your notes — and who reports back only the conclusion.

When They Shine — and When They Don't

Subagents help when subtasks are:

- **Parallelizable** — independent work that doesn't need to be sequenced
- **Bounded** — clear inputs, clear outputs, no back-and-forth
- **Independently verifiable** — you can check the result without needing the agent's full trace

Research workflows that fit:

- Multi-database literature searches (one agent per database)
- Coding many documents against a fixed rubric
- Parallel robustness checks on independent specifications

Research workflows that don't fit:

- Anything sequential and judgment-heavy — most of your drafting, most of your analysis decisions
- Tasks that depend on your ongoing conversation context (subagents start cold)

The Honest Teaching

Most research tasks do not have the shape that makes subagents worth it.

- Most of what you do is sequential
- Most of your decisions depend on context the subagent won't see
- Most of your verification needs the full trace, not a summary

If your workflow has parallel, independent, verifiable subtasks — revisit this. Otherwise, don't over-engineer.

The meta-lesson, one more time

Subagents are genuinely useful for a specific shape of problem. They're also the shiniest thing in the toolkit right now. Asking *"does my problem have that shape?"* is the whole game.

What You Built Today

1. **A cleaner workflow.** You're working in your own workspace. No more merge conflicts against the shared repo.
2. **A framework for skills.** A routing rule (stable-about-you / within-a-paper / live) plus the honest view: skills are empirical, and most get abandoned.
3. **Two MCPs on your map.** Zotero (friction) and GitHub (capability). The two-reason rule to evaluate others.
4. **An honest view of subagents.** Know they exist. Know the shape of problem they solve. Don't reach for them until your problem has that shape.

The thread running through

Agents solve symptoms well. Your job is to notice when the structure is wrong, when the skill is ceremony, when the MCP is novelty, and when the subagent is overkill.

Next Session + Prework

Session 5: Expert Judgment and Capstone.

- Failure patterns — what goes wrong and how to recognize it
- When to correct the approach, restart the conversation, or do it yourself
- Capstone: build an end-to-end research artifact using everything from Sessions 1–4

Before next session:

1. **Install the Zotero MCP** (setup guide linked in the course README)
2. **Run the routing test** on one thing you wished Claude already knew — is it a skill, project context, or MCP?
3. **Try one thing from today** on your actual research: a skill, an MCP, or just the new repo workflow

Bring a question, a frustration, or a small win from the week. Session 5 opens with those.